

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	G.Dhanusha Sree	Reg. No.:	192325012
Section / Batch:	2023	Date:	14/07/2026

Code of Conduct

I G.Dhanusha Sree(192325012) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G.Dhanusha Sree

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

Q1. Resume Information Extraction using Regular Expressions

Problem Understanding

A recruitment company receives thousands of resumes every day. Manually checking each resume is time-consuming and may lead to errors. A Python program using Regular Expressions can automatically extract important candidate details such as name, email, phone number, technical skills, and years of experience. The system also checks whether the candidate is eligible for shortlisting based on predefined criteria.

Approach

- Read the resume as text.
- Use Regular Expressions (re module) to extract:
 - Candidate name
 - Email address
 - Mobile number
 - Technical skills
 - Years of experience
- Generate a structured candidate summary.
- Check eligibility:
 - Experience $\geq$ 2 years
 - Python skill should be present.

Python Program

```
import re

resume = """
Name: Dhanusha Sree
Email: dhanusha@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""

# Name
name = re.search(r"Name:\s*(.*)", resume).group(1)

# Email
email = re.search(r"\b[\w\.-]+@[\w\.-]+\.\w+\b", resume).group()
```

```

# Mobile
mobile = re.search(r"\b\d{10}\b", resume).group()

# Skills
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found_skills = [skill for skill in skills if re.search(skill, resume, re.I)]

# Experience
experience = int(re.search(r"(\d+)\s+years", resume).group(1))

print("Candidate Summary")
print("-----")
print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", ", ".join(found_skills))
print("Experience:", experience, "years")

if experience >= 2 and "Python" in found_skills:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")

```

Output

```

Candidate Summary
-----
Name: Dhanusha Sree
Email: dhanusha@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
Eligible for Shortlisting

```

Explanation

- Extracts candidate name.
- Finds email and mobile number.
- Detects technical skills.
- Extracts years of experience.
- Displays candidate summary.
- Checks eligibility (minimum 2 years experience and Python skill).

Section 2: Regular Expressions – Product Search System

Q2. Product Search System using Regular Expressions

Problem Understanding

An e-commerce company wants to improve its product search engine so customers can easily find products using different search methods. Regular Expressions allow flexible searching such as exact matching, prefix matching, suffix matching, partial matching, and case-insensitive searching.

Approach

- Store product names in a list.
- Accept a search keyword from the user.
- Perform:
 - Exact search
 - Prefix search
 - Suffix search
 - Partial search
 - Case-insensitive search
- Display all matching products.
- Generate a report showing the number of matched products.

Python Program

```
import re

products = [
    "Laptop",
    "Laptop Bag",
    "Mouse",
    "Keyboard",
    "Laptop Stand",
    "Gaming Mouse",
    "Monitor"
]

keyword = input("Enter search keyword: ")

exact = [p for p in products if re.fullmatch(keyword, p, re.I)]
prefix = [p for p in products if re.match(keyword, p, re.I)]
suffix = [p for p in products if re.search(keyword + r"$", p, re.I)]
partial = [p for p in products if re.search(keyword, p, re.I)]

print("\nExact Match:", exact)
print("Prefix Match:", prefix)
print("Suffix Match:", suffix)
print("Partial Match:", partial)
```

```
print("\nReport")
print("Exact:", len(exact))
print("Prefix:", len(prefix))
print("Suffix:", len(suffix))
print("Partial:", len(partial))
```

Sample Input

Laptop

Output

```
Exact Match: ['Laptop']
Prefix Match: ['Laptop', 'Laptop Bag', 'Laptop Stand']
Suffix Match: ['Laptop']
Partial Match: ['Laptop', 'Laptop Bag', 'Laptop Stand']
```

```
Report
Exact: 1
Prefix: 3
Suffix: 1
Partial: 3
```

Explanation

- Searches exact product name.
- Searches products starting with keyword.
- Searches products ending with keyword.
- Searches products containing keyword.
- Supports case-insensitive search.
- Displays matching products and report.

Section 3: Regular Expressions – University Registration System

Q3. University Registration Validation System

Problem Understanding

A university requires students to enter valid details during online registration. Incorrect information may lead to registration errors. A Regular Expression-based validation system checks whether all entered details follow the required format before allowing successful registration.

Approach

- Accept student details:
 - Register Number
 - Institutional Email
 - Course Code
 - Semester
 - Mobile Number
- Validate each field using Regular Expressions.
- Display whether each input is valid or invalid.
- Generate the final registration status.

Python Program

```
import re

reg = input("Enter Register Number: ")email = input("Enter Email: ")course =
input("Enter Course Code: ")semester = input("Enter Semester: ")mobile =
input("Enter Mobile Number: ")
reg_valid = re.fullmatch(r"\d{12}", reg)email_valid = re.fullmatch(r"[\w\.-
]+@saveetha\.com", email)course_valid = re.fullmatch(r"[A-Z]{3}\d{2}",
course)semester_valid = re.fullmatch(r"[1-8]", semester)mobile_valid =
re.fullmatch(r"[6-9]\d{9}", mobile)
print("\nValidation Results")
print("Register Number:", "Valid" if reg_valid else "Invalid")print("Email:", "Valid"
if email_valid else "Invalid")print("Course Code:", "Valid" if course_valid else
"Invalid")print("Semester:", "Valid" if semester_valid else "Invalid")print("Mobile
Number:", "Valid" if mobile_valid else "Invalid")
if all([reg_valid, email_valid, course_valid, semester_valid, mobile_valid]):
    print("\nRegistration Successful")else:
    print("\nRegistration Failed")
```

Sample Input

Register Number: 202312345678
Email: student@saveetha.com
Course Code: NLP03
Semester: 5
Mobile Number: 9876543210

Output

Validation Results
Register Number: Valid
Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

Registration Successful

Explanation

- Validates 12-digit register number.
- Checks institutional email ending with @saveetha.com.
- Validates course code format (e.g., NLP03).
- Accepts semester values from 1–8.
- Validates 10-digit Indian mobile number.
- Displays validation messages and final registration status.